

Multi-Factor Stock Selection System

Core Algorithms & Data Flow

Rank IC Analysis - Factor Optimization - Backtesting
Stock Scoring - ML Training - Portfolio Construction

1. End-to-End Data Flow

The complete pipeline from raw market data to backtest results:

- [1] DATA ACQUISITION
Tushare/Baostock/SQL dumps -> stock_daily_history (1.47M), stock_daily_basic (376K), stock_factor (1.13M), stock_moneyflow (459K), stock_cyq_perf (218K)
- [2] FACTOR COMPUTATION (scripts/calculate_factors.py)
Price data -> pandas vectorized ops -> 6 technical factors
factor_values: 8.37M rows, 272 trading days, 5,473 stocks
- [3] FACTOR OPTIMIZATION (FactorOptimizer)
factor_values + prices -> Rank IC (Spearman) -> Rolling IC stats (IC_mean, IC_IR, win_rate) -> Factor selection (threshold + redundancy) -> Weight calculation (|IC_IR| proportional) -> factor_effectiveness table
- [4] STOCK SCORING (StockScoringEngine)
factor_values + optimized_weights -> composite_score per stock
-> rank_stocks -> Top N stocks with basic info
- [5] BACKTEST (BacktestEngine)
Monthly rebalance -> select -> optimize -> execute -> track P&L
-> metrics (total_return, sharpe, max_drawdown, win_rate, calmar_ratio)

2. Rank IC Algorithm Deep Dive

2.1 Mathematical Definition

$\text{Rank IC}(t, f, N) = \text{SpearmanRho}(f_value(t), \text{forward_return}(t, t+N))$

where:

$f_value(t)$ = cross-sectional factor values for all stocks on date t
 $\text{forward_return}(t, t+N) = (\text{close}(t+N) - \text{close}(t)) / \text{close}(t)$ for each stock
 SpearmanRho = Pearson correlation on rank-transformed data

Spearman is used instead of Pearson because stock factor values and returns are typically non-normally distributed. Rank-based correlation is distribution-free and robust to outliers.

2.2 Single-Day IC Computation (Vectorized)

```
# Step 1: Get factor exposure for ALL stocks on date T
factor_df = factor_engine.get_factor_exposure(factor_id, trade_date)
# Returns DataFrame: ts_code, factor_value, z_score, percentile_rank

# Step 2: Get current close prices (single query, 5473 rows)
current_data = pd.read_sql(
    StockDailyHistory.query.filter(trade_date == trade_dt).statement, db.engine)

# Step 3: Get future prices (single query, all future data)
price_data = pd.read_sql(
    StockDailyHistory.query.filter(trade_date > trade_dt).statement, db.engine)

# Step 4: For each stock, get the forward_period-th future close
# Using pandas groupby.cumcount for vectorized indexing (avoids per-stock loop)
position_counts = price_data.groupby('ts_code').cumcount()
future_prices = price_data[position_counts == (forward_period - 1)][['ts_code', 'close']]

# Step 5: Merge and compute
merged = pd.merge(current_data[['ts_code', 'close']], future_prices, on='ts_code')
merged['forward_return'] = (merged['close_y'] - merged['close_x']) / merged['close_x']

# Step 6: Spearman rank correlation
final = pd.merge(factor_df[['ts_code', 'factor_value']], merged[['ts_code', 'forward_return']])
ic, p_value = spearmanr(final['factor_value'], final['forward_return'])
```

2.3 Rolling Statistics Aggregation

Once single-day ICs are computed for all dates in the evaluation window:

```
IC_mean = (1/n) * sum(IC_i)          # Average prediction ability
IC_std  = sqrt((1/(n-1)) * sum(IC_i - IC_mean)^2) # Stability
IC_IR   = IC_mean / IC_std           # Information Ratio
IC_win_rate = count(IC_i > 0) / n    # Consistency
```

IC_IR is the primary metric because it captures both magnitude and consistency. A factor with IC_mean=0.05 and IC_std=0.02 (IC_IR=2.5) is preferred over one with IC_mean=0.10 and IC_std=0.10 (IC_IR=1.0) despite the latter having higher average IC.

3. Factor Selection Algorithm

3.1 Stage 1: IC_IR Threshold

```
threshold = 0.3 # default, user-configurable
valid_factors = []
for f in all_factors:
    if f.ic_ir is not None and abs(f.ic_ir) > threshold:
        valid_factors.append(f)

# If too few factors pass, relax threshold and take top N
if len(valid_factors) < min_factors:
    valid_factors = sorted(all_factors, key=abs(ic_ir))[-max(min_factors, max_factors):]
```

3.2 Stage 2: Correlation Redundancy Removal

```
# Compute cross-sectional factor correlation matrix
pivot = factor_values.pivot(index='ts_code', columns='factor_id', values='factor_value')
corr_matrix = pivot.corr(method='spearman')

# Greedy selection: pick highest |IC_IR| first, skip correlated
selected = []
for f in sorted(valid_factors, key=lambda x: abs(x.ic_ir), reverse=True):
    redundant = False
    for s in selected:
        if abs(corr_matrix.loc[s.id, f.id]) > max_correlation:
            redundant = True; break
    if not redundant:
        selected.append(f)
```

3.3 Weight Methods

- `ic_ir_weighted` (default): $\text{weight}_i = |\text{IC_IR}_i| / \sum(|\text{IC_IR}_j|)$. Favors stable predictive factors.

- `ic_mean_weighted`: $\text{weight}_i = |\text{IC_mean}_i| / \sum(|\text{IC_mean}_j|)$. Favors strong raw signal.
- `equal_weight`: $\text{weight}_i = 1/n$. Useful as baseline comparison.
- Weights always normalized to $\sum=1.0$. Factors with IC near zero get near-zero weight.

4. Stock Scoring Pipeline

```
# 1. Load factor Z-scores for trade_date
factor_scores = pd.pivot_table(
    FactorValues.query.filter(trade_date=T, factor_id in factor_list),
    index='ts_code', columns='factor_id', values='z_score'
).fillna(0) # shape: (5473 stocks, N factors)

# 2. Apply weights via scoring method
if method == 'rank_ic':
    weights = FactorOptimizer.get_optimized_weights(evaluation_date=T)
else:
    weights = user_provided or equal_weights

composite_scores = (factor_scores * weights).sum(axis=1)

# 3. Rank and select top N
result = composite_scores.sort_values(ascending=False).head(top_n)

# 4. Enrich with stock info (name, industry, area)
stock_info = StockBasic.query.filter(ts_code in result.index)
```

5. Backtest Algorithm

```
# Monthly rebalance backtest
positions, cash, portfolio_values = {}, initial_capital, []

for trade_date in monthly_dates(start, end):
    # 1. Select stocks for this rebalance date
    selected = scoring_engine.calculate_factor_scores(trade_date, factor_list)
    composite = scoring_engine.calculate_composite_score(selected, weights, method)
    top_stocks = scoring_engine.rank_stocks(composite, top_n)
    target_weights = optimizer.optimize_portfolio(expected_returns)

    # 2. Get current prices for holdings + targets
    prices = get_current_prices(trade_date, positions.keys() | target_weights.keys())

    # 3. Calculate current portfolio value
    current_value = sum(positions[s] * prices[s] for s in positions) + cash

    # 4. Execute rebalance with transaction costs
    new_positions, new_cash, turnover = rebalance(
        positions, cash, target_weights, prices, current_value, tx_cost
    )

    # 5. Record portfolio value
    positions, cash = new_positions, new_cash
    portfolio_values.append({'date': trade_date, 'total_value': current_value})

# 6. Compute performance metrics
daily_returns = [(pv[i+1].total_value / pv[i].total_value - 1) for i in range(len(pv)-1)]
total_return = (portfolio_values[-1].total_value / initial_capital) - 1
annualized_return = (1 + total_return) ^ (252 / len(daily_returns)) - 1
sharpe_ratio = mean(daily_returns) / std(daily_returns) * sqrt(252)
```

6. ML Model Training Pipeline

```
# 1. Prepare training data
factor_data = pd.pivot_table(
    FactorValues.query.filter(factor_id in model_def.factor_list,
                             trade_date between start and end),
    index=['ts_code', 'trade_date'], columns='factor_id', values='factor_value'
)
# Compute target: forward N-day return
target = calculate_forward_returns(factor_data, model_def.target_type) # return_5d, return_20d

# 2. Feature engineering
X = RobustScaler().fit_transform(factor_data[model_def.factor_list])
if feature_selection_enabled:
    X, selected = SelectKBest(f_regression, k=min(k, n_features)).fit_transform(X, y)

# 3. Time-series split (preserve temporal order, no shuffle)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

# 4. Train and evaluate
model = RandomForestRegressor(n_estimators=100, max_depth=10, n_jobs=-1)
model.fit(X_train, y_train)

# 5. Cross-validation with TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
cv_scores = cross_val_score(model, X, y, cv=tscv, scoring='r2')

# 6. Persist model and feature importance
joblib.dump(model, f'models/{model_id}.pkl')
model_def.feature_importance = dict(zip(features, model.feature_importances_))
db.session.commit()
```

7. Portfolio Optimization Algorithms

```
# Mean-Variance Optimization (CVXPY)
w = cp.Variable(n)
expected_return = mu @ w
risk = cp.quad_form(w, Sigma)
objective = cp.Maximize(expected_return - 0.5 * risk_aversion * risk)
constraints = [cp.sum(w) == 1, w >= 0] # long-only, fully invested
problem = cp.Problem(objective, constraints)
problem.solve(solver='ECOS')

# Risk Parity (SciPy)
def risk_parity_objective(w):
    portfolio_vol = sqrt(w.T @ Sigma @ w)
    marginal_risk = Sigma @ w
    risk_contrib = w * marginal_risk / portfolio_vol
    target_contrib = portfolio_vol / n # equal risk contribution
    return sum((risk_contrib - target_contrib)**2)

result = minimize(risk_parity_objective, x0=ones(n)/n,
                  constraints={'type': 'eq', 'fun': lambda x: sum(x)-1},
                  bounds=[(0,1)]*n)
```

8. Performance Metrics Reference

- Total Return: $(\text{final_value} / \text{initial_capital}) - 1$
- Annualized Return: $(1 + \text{total_return})^{(252/\text{trading_days})} - 1$
- Sharpe Ratio: $\text{mean}(\text{daily_returns}) / \text{std}(\text{daily_returns}) * \text{sqrt}(252)$
- Max Drawdown: max peak-to-trough decline in portfolio value
- Calmar Ratio: $\text{annualized_return} / |\text{max_drawdown}|$
- Volatility: $\text{std}(\text{daily_returns}) * \text{sqrt}(252)$ (annualized)
- Win Rate: percentage of rebalance periods with positive returns
- Turnover: percentage of portfolio value traded each rebalance

9. IC Analysis - Real Data Results

Analysis period: 2025-01-03 ~ 2025-04-15, 66 trading days, 6 technical factors:

Factor	IC_mean	IC_IR	WinRate	Weight	Selected?
volatility_20d	-0.1566	-0.64	29%	42.3%	YES
momentum_5d	-0.0533	-0.45	26%	30.1%	YES
momentum_20d	-0.0376	-0.24	39%	15.7%	YES
momentum_1d	-0.0289	-0.18	40%	12.0%	YES
volume_ratio_20d	-0.0015	-0.02	50%	0%	NO (IC_IR too low)
price_to_ma20	-0.0115	-0.16	47%	0%	NO (redundant with momentum)

Key finding: All factors showed NEGATIVE IC in this period, indicating a mean-reversion market. The optimization engine correctly identified volatility_20d as the strongest predictor and assigned it the highest weight (42.3%).

10. Backtest Results (Real Data)

Full period backtest: 2025-01-03 ~ 2026-02-12, 14 monthly rebalances, 20-stock portfolio:

Total Return: +2.96%
 Annualized Return: +2.66%
 Sharpe Ratio: -0.003
 Max Drawdown: -23.2% (March-April 2025 crash)
 Win Rate: 53.8% (7/14 months positive)
 Calmar Ratio: 0.11

Monthly Performance:

Jan 2025: 998k Feb: 986k Mar: 1,140k (+15.6%, best)
 Apr: 931k May: 875k Jun: 928k
 Jul: 934k Aug: 940k Sep: 1,027k
 Oct: 1,090k Nov: 1,112k Dec: 1,097k
 Jan 2026: 1,046k Feb: 1,030k

Notes:

- Look-ahead bias exists: all months use the same global IC weights
- Early months (Jan-Feb) have incomplete factor data (1-2 factors only)
- 23% max drawdown suggests excessive exposure to small-cap volatile stocks
- Strategy needs more factor types (fundamental, money flow) for diversification

11. System Configuration

```
# config.py - Key settings
class Config:
    SECRET_KEY = 'dev-secret-key'
    SQLALCHEMY_DATABASE_URI = 'mysql+pymysql://root:123456@localhost/stock_cursor'
    REDIS_URL = 'redis://localhost:6379/0'

# Database: MySQL 8.0 (stock_cursor)
# Redis: localhost:6379 (cache and message queue)
# LLM: OpenAI-compatible API, default Aliyun DashScope qwen-coder-turbo
# WebSocket: eventlet async mode, port 5001
# Logging: loguru -> logs/stock_analysis.log
```

12. Startup & Deployment

```
# Development startup
python run.py                # Flask dev server on port 5001
python run_system.py        # Interactive system manager (check deps, init DB, start)

# Data import
mysql -uroot -p < stock_data/stock_basic.sql
mysql -uroot -p < stock_data/stock_daily_history.sql # 336MB
# ... (8 SQL files total)
python scripts/fast_import.py # Optimized batch import (10k rows/txn)

# Factor computation
python scripts/calculate_factors.py # 272 days x 6 factors -> 8.37M rows

# Factor optimization (via API or direct)
POST /api/ml-factor/factors/optimize
# or: FactorOptimizer.get_optimized_weights(evaluation_date)

# Backtest (via API or UI)
POST /api/ml-factor/backtest/run
# or: http://127.0.0.1:5001/ml-factor/backtest
```